AWS Level 1 EC2 SSH Keychain and Nginx

Student: Claudio Verdi

Implementation date: 23 October 2026

Evidence: submitted command history and screenshots from the implementation.

Purpose and preparation

The purpose was to create an EC2 web server manually, connect from a local Ubuntu terminal using an SSH key, install Nginx and Git, and publish the supplied HTML page. The implementation followed the supplied EC2 Level 1 Keychain material. Commands were performed individually; the instructor's sample output was not treated as evidence of this implementation.

Keychain manages the local SSH agent and loaded identities. SSH authenticates the client using a private key whose corresponding public key must be authorized on the server. Nginx serves the deployed HTML file over HTTP. Git retrieves the supplied website repository.

Environment

Setting	Value
AWS region	Europe (Stockholm), eu-north-1
Instance name	level1-devops-jan-2026
Instance ID	i-0c1a003d2ee24806f
Instance type	t3.micro
Operating system	Amazon Linux 2023
Storage	8 GiB gp3
Public IPv4 at verification	13.60.77.43
Private IPv4	172.31.46.57
SSH user	ec2-user
Local key path	~/.ssh/ec2kp
Security group	launch-wizard-6

SSH access was restricted to My IP during instance creation, and HTTP access was allowed from the internet. The instance was running and passed its status checks. The public IP recorded here may change after stopping and starting the instance.

Local Keychain setup and SSH troubleshooting

Keychain was already installed locally. The key was loaded using:

```
eval "$(keychain --eval ~/.ssh/ec2kp)"
```

ssh-add -l

The Keychain initialization was added to the local ~/.bashrc file. Initial SSH attempts returned Permission denied (publickey,gssapi-keyex,gssapi-with-mic).

Both local key files, ec2kp and ec2kp.pem, represented the same private key. Comparing fingerprints using the same MD5 format showed that the local key differed from the AWS key pair associated with the instance. The local fingerprint was 77:2a:a6:1c:5b:41:db:b8:bb:a5:d0:4b:66:1b:2a:ff.

The problem was resolved through an EC2 Instance Connect browser session. The local public key was derived with ssh-keygen -y -f ~/.ssh/ec2kp and appended to the remote ~/.ssh/authorized_keys file. The private key remained on the local computer. The initial paste did not save the key, so the append operation was repeated and verified using ssh-keygen -lf ~/.ssh/authorized_keys.

The following command then successfully opened an Amazon Linux session:

ssh -o IdentitiesOnly=yes -i ~/.ssh/ec2kp ec2-user@13.60.77.43

Local public key authorized on the server

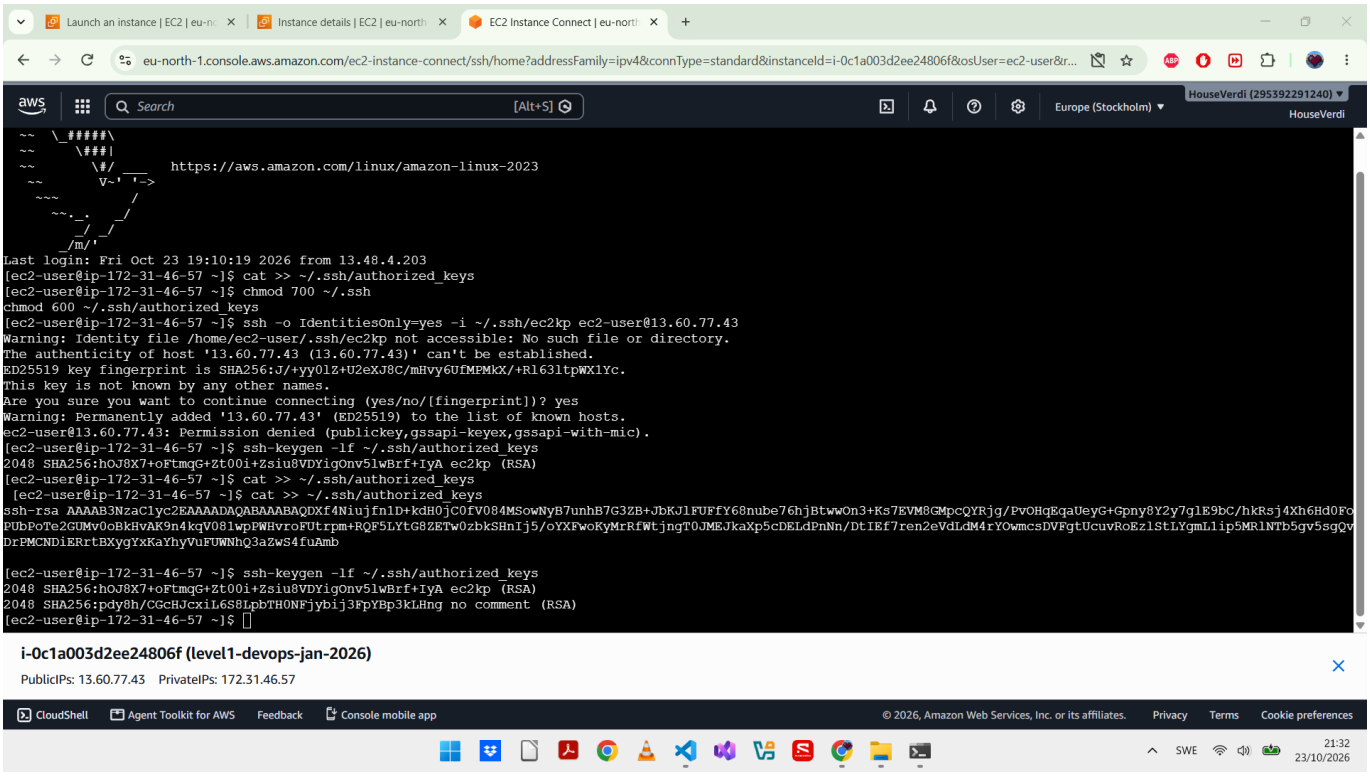


Figure 1. Local public key authorized on the server. Source: image(20261023-193211).png

Successful SSH connection

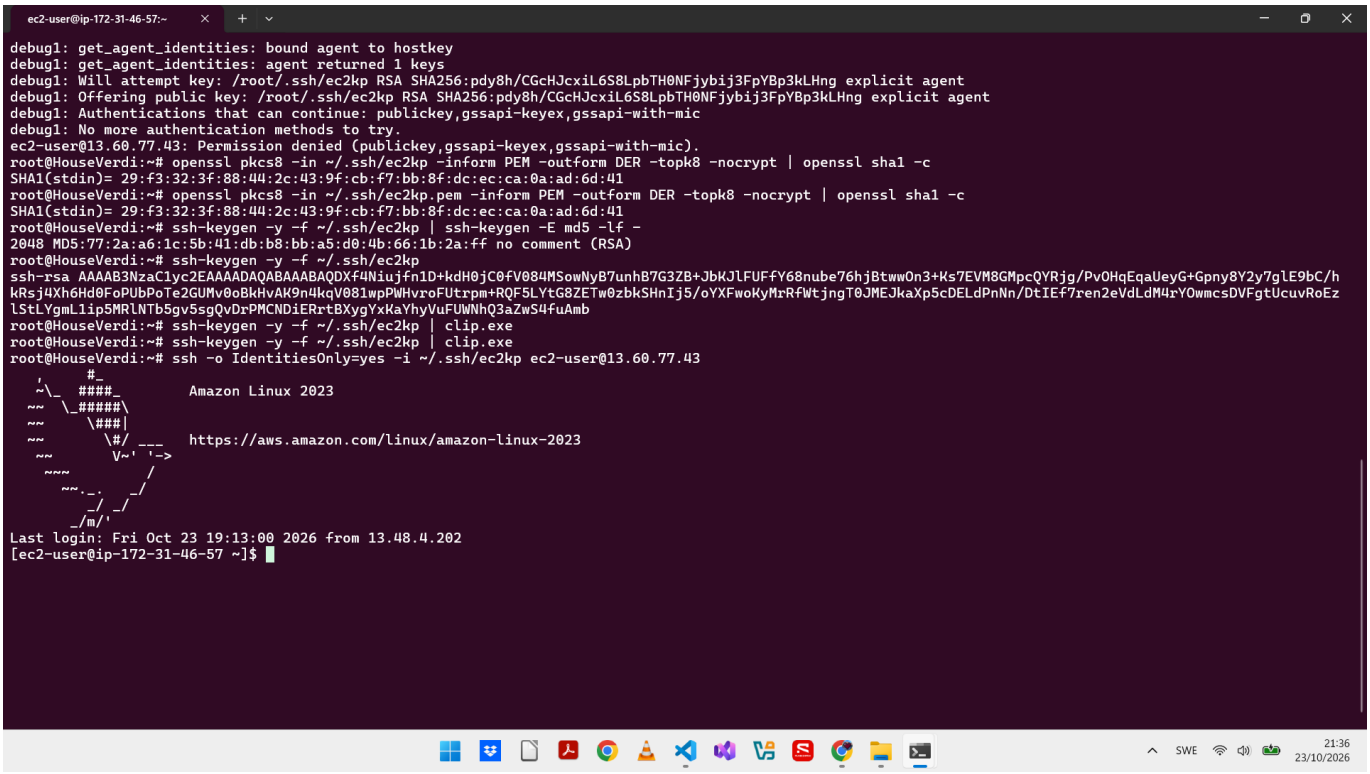


Figure 2. Successful SSH connection. Source: image(20261023-193649).png

Installation

After connecting, the following commands were executed. The exported root command history confirms this sequence:

sudo bash
yum update -y
yum upgrade -y
yum install nginx -y
service nginx start
yum install git-core -y
exit

The Nginx installation screenshot showed successful completion. Starting Nginx through the service command redirected to systemctl without an error. The SSH session was closed and reopened before deployment, as in the supplied workflow.

Nginx installed and started

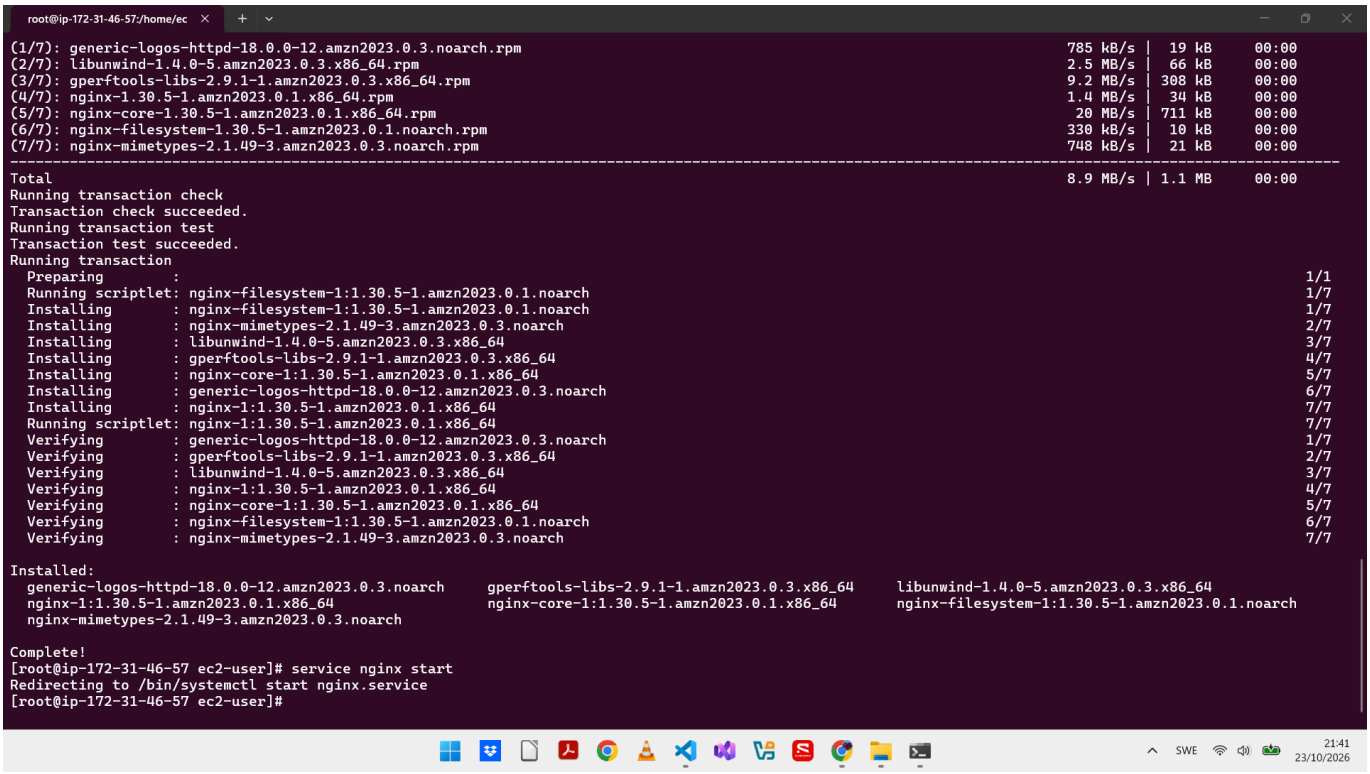


Figure 3. Nginx installed and started. Source: image(20261023-194113).png

Website deployment

The supplied repository was cloned and inspected:

git clone https://github.com/nenemustafa/Hellome.git

ls -la Hellome

sudo cp Hellome/heyme /usr/share/nginx/html/index.html

The directory listing confirmed that heyme was a regular file of 499 bytes. It was copied to index.html in the Nginx document root. Copying retained the repository's original file while publishing the page. The supplied page is instructor-provided content; this exercise demonstrates server setup and deployment.

Repository cloned and files inspected

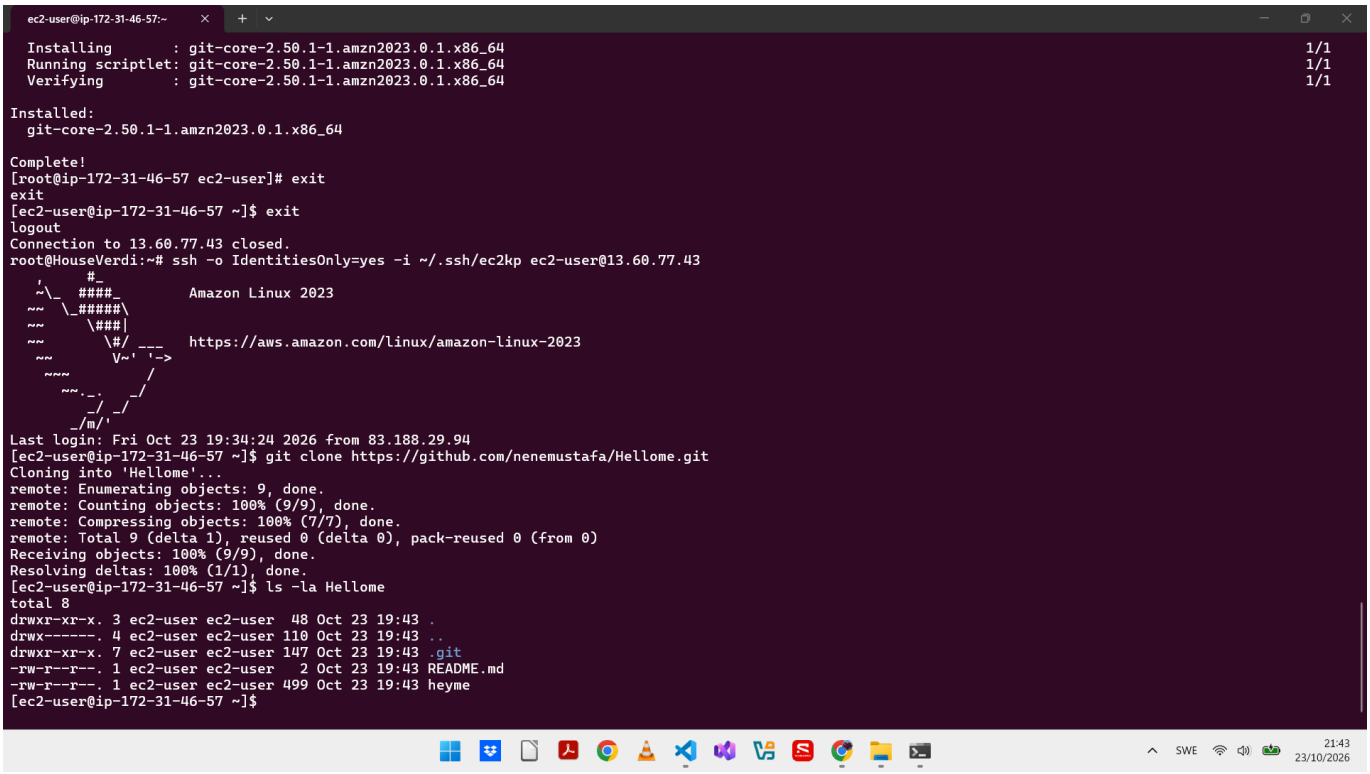


Figure 4. Repository cloned and files inspected. Source: image(20261023-194347).png

Verification and improvement

curl -I http://localhost

systemctl status nginx --no-pager

sudo systemctl enable nginx

systemctl is-enabled nginx

Check	Observed result
Local HTTP request	HTTP/1.1 200 OK
Server header	nginx/1.30.5
Content type	text/html
Content length	499 bytes
Nginx service	active (running)
Configuration check in service log	syntax is ok; test is successful
Automatic startup after enabling	enabled
External browser check	“Hey me!” page displayed at http://13.60.77.43

The initial service status showed automatic startup as disabled. Enabling the service was an additional improvement beyond manually starting it. The enabled setting was verified; a reboot test was not performed. The page was served over HTTP, so HTTPS was not configured in this exercise.

Published website in the browser

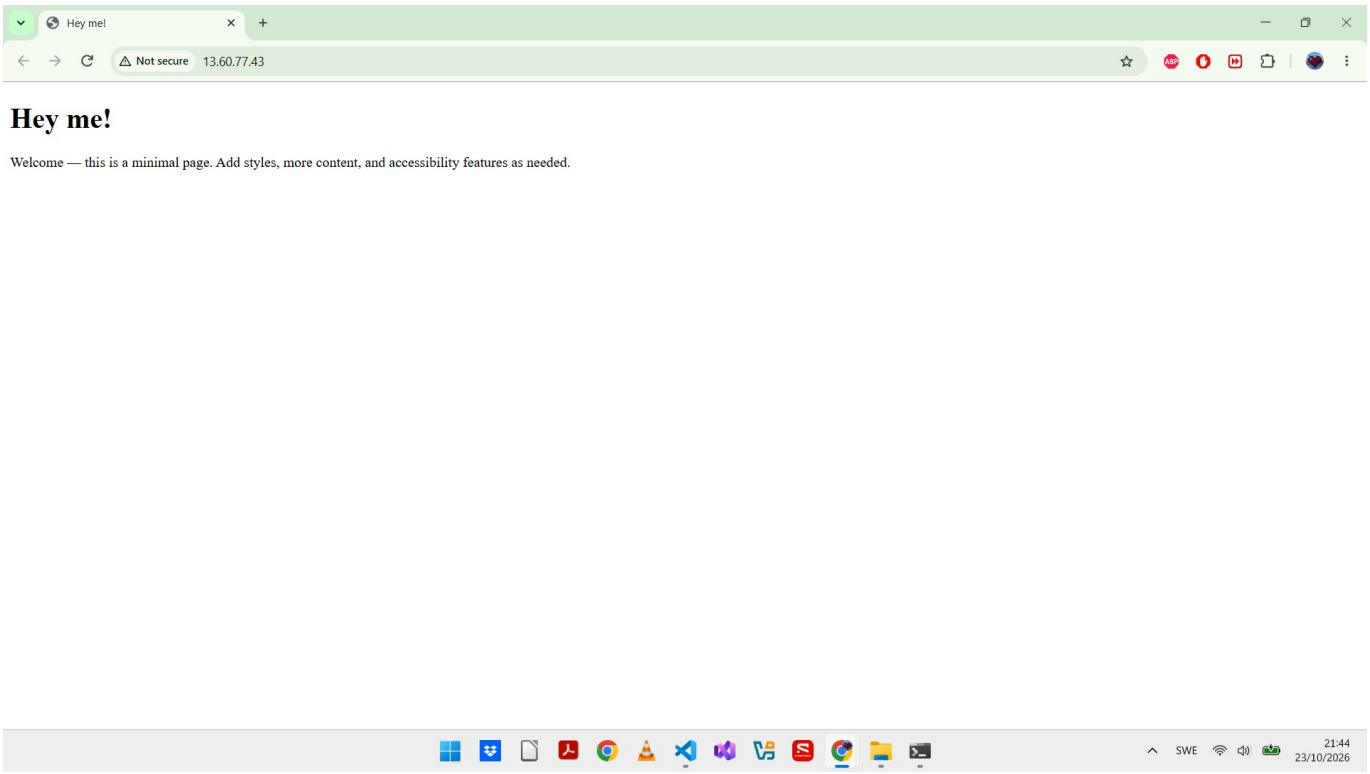


Figure 5. Published website in the browser. Source: image(20261023-194449).png

HTTP 200 response from Nginx

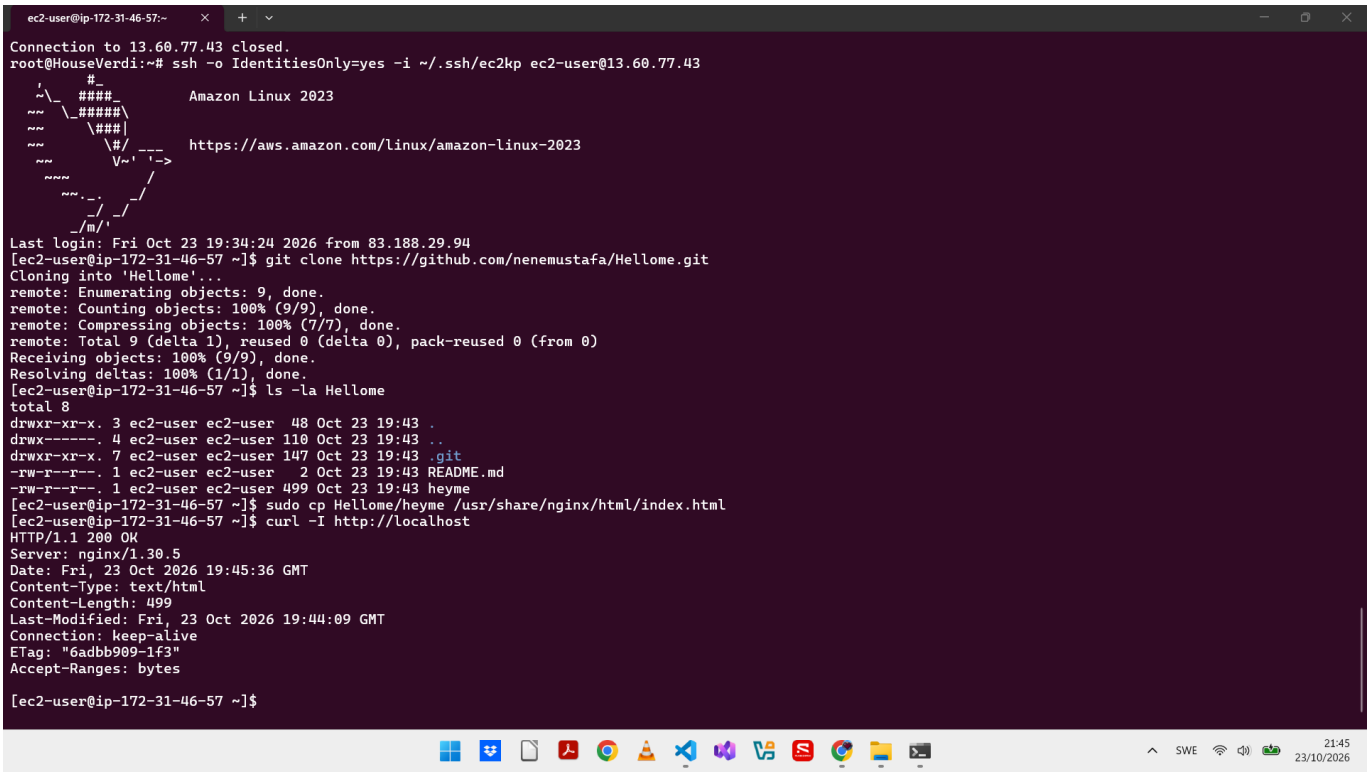


Figure 6. HTTP 200 response from Nginx. Source: image(20261023-194546).png

Nginx is running

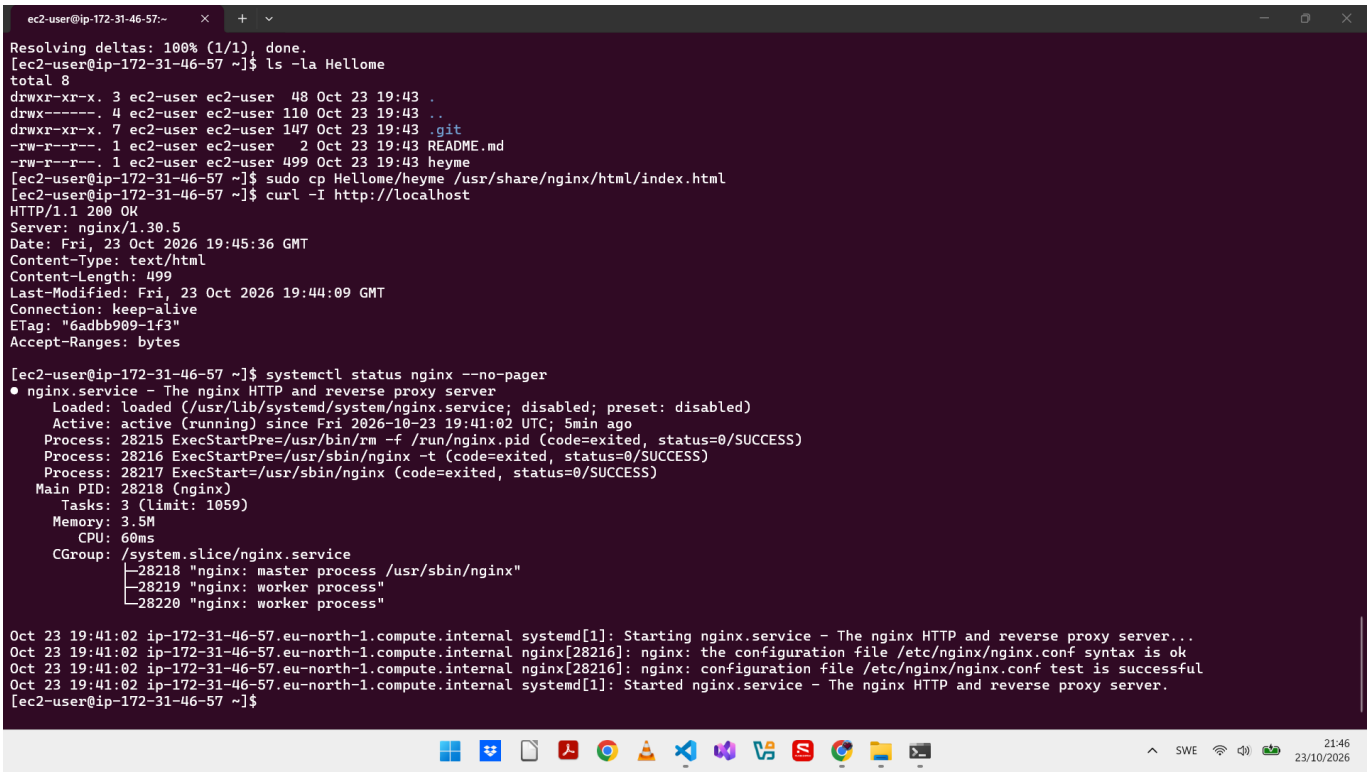


Figure 7. Nginx is running. Source: image(20261023-194617).png

Automatic startup enabled

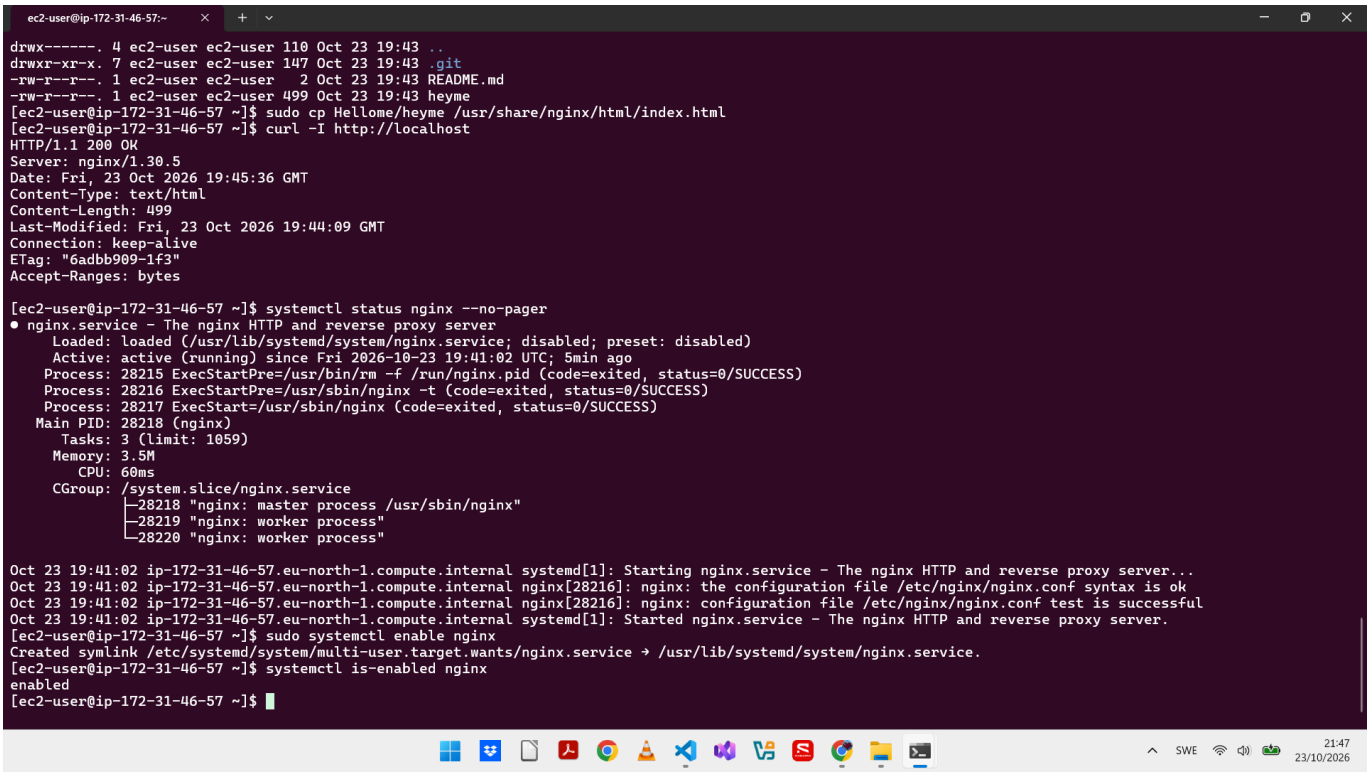


Figure 8. Automatic startup enabled. Source: image(20261023-194708).png

Evidence record

The following submitted screenshots provide evidence alongside level1-command-history.txt:

Screenshot filename	Evidence
image(20261023-193211).png	Local public key present in authorized_keys
image(20261023-193649).png	Successful SSH connection from the local terminal
image(20261023-194113).png	Nginx installation completed and start command executed
image(20261023-194347).png	Repository cloned and heyme file listed
image(20261023-194449).png	Published page displayed in the browser
image(20261023-194546).png	Deployment command and HTTP 200 response
image(20261023-194617).png	Nginx active and configuration check successful
image(20261023-194708).png	Automatic startup enabled
image(20261023-194747).png	ec2-user command history
image(20261023-194822).png	Root installation command history

Screenshots are embedded next to the relevant stages and identified by their original filenames. Command history records commands rather than their full output; the screenshots supply the observed results. No terminal output has been reconstructed as an original session transcript.

Command history for ec2-user

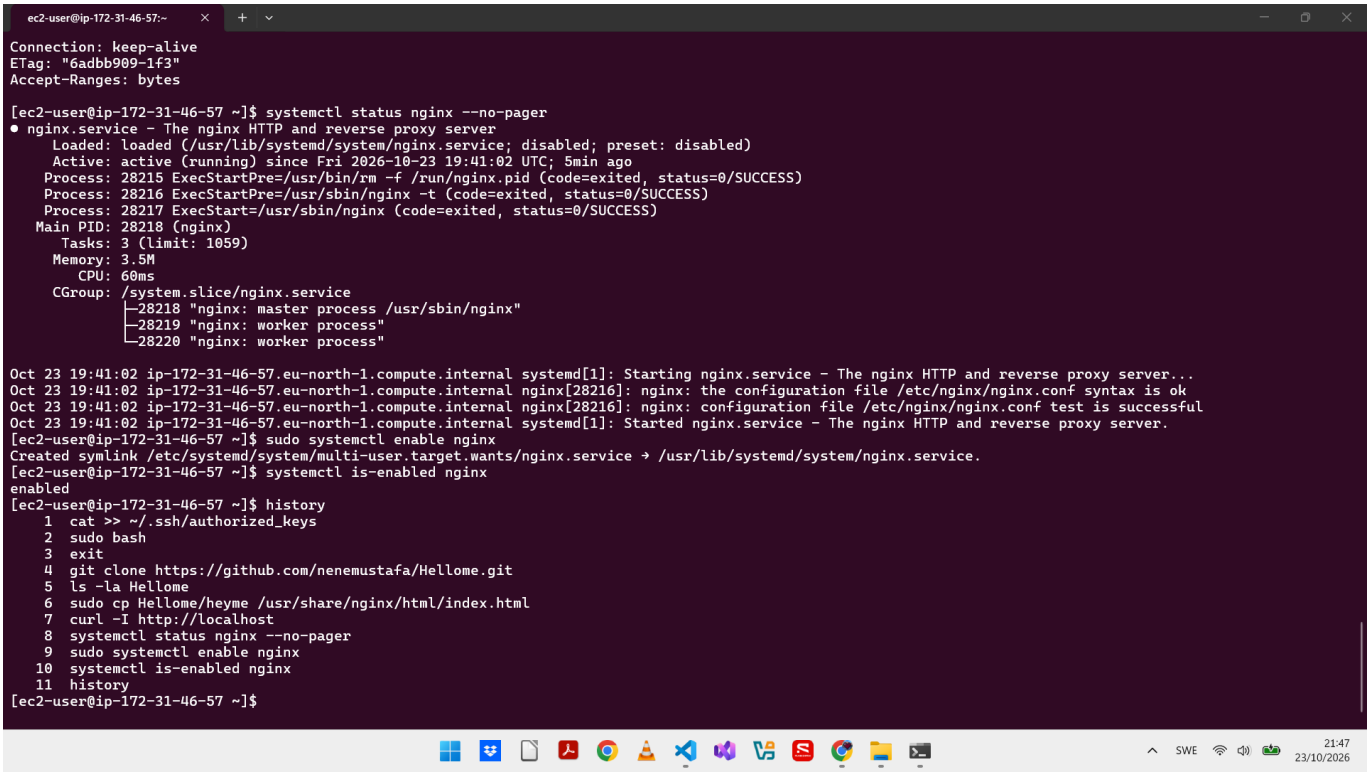


Figure 9. Command history for ec2-user. Source: image(20261023-194747).png

Root installation commands

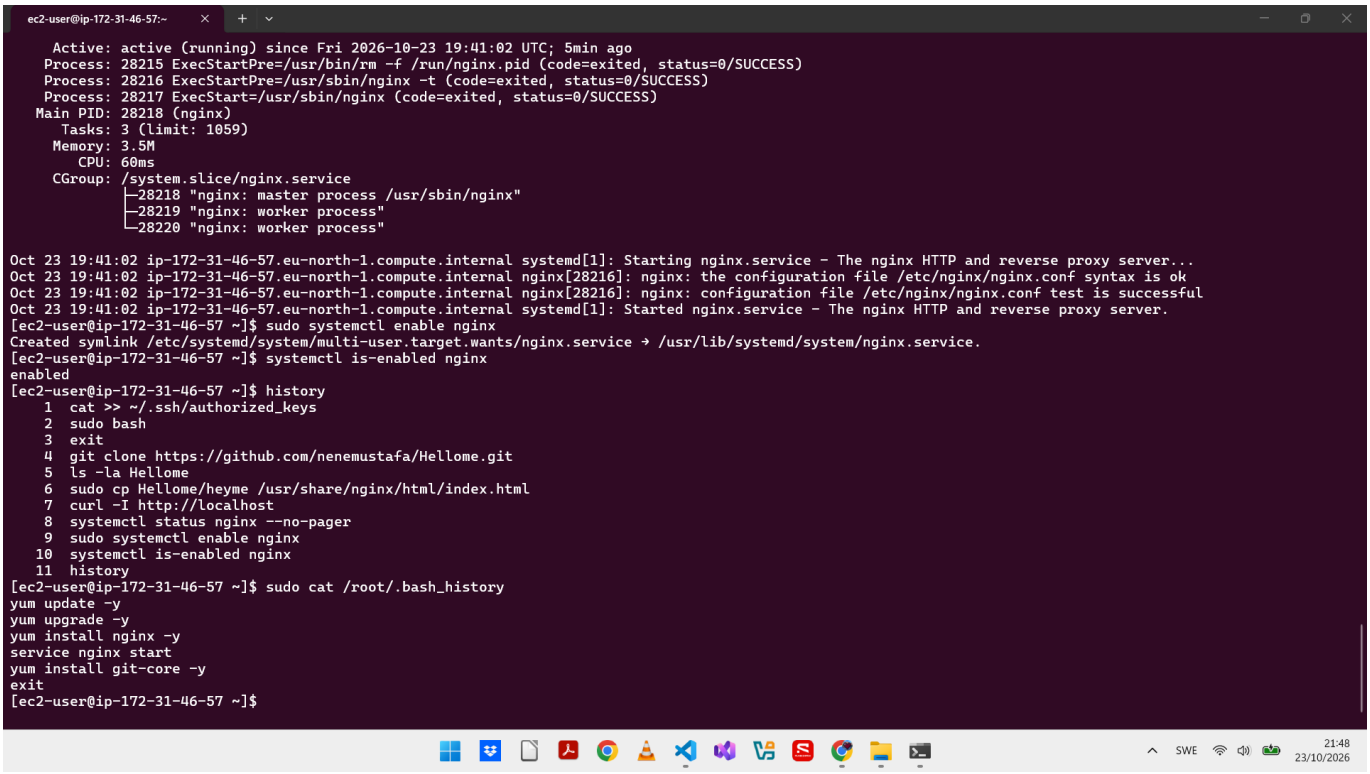


Figure 10. Root installation commands. Source: image(20261023-194822).png

Conclusion

The Level 1 implementation produced a reachable EC2 web server running Nginx. SSH authentication was repaired by authorizing the correct local public key. The supplied website was deployed successfully and verified through an HTTP 200 response, service status and a browser check. Nginx was also configured to start automatically. Possible future improvements include HTTPS and verifying service availability after a reboot.

Reference material

- EC2-Level1-Keychain-Complete (2)(2).sh: instructions for creating an EC2 instance, configuring Keychain and SSH, installing Nginx and Git, deploying the website and verifying the result.
- Student evidence: level1-command-history.txt and the screenshots listed above.